JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

IES ALBARREGAS

TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

DEPARTAMENTO DE INFORMÁTICA

Saniclear

Saniclear

MANUAL TÉCNICO

Autor/es: María Ceballos Mesias

Curso Académico: 2025/26

ÍNDICE

CONTENIDO

1.Introducción 3
1.1. descripción del proyecto 3
1.2 Justificacción y Problemática 3
1.3 Ámbito del Proyecto 3
1.4 Diagrama de Contexto 5
2. Arquitectura de la aplicación 5
2.1 fronted 6
2.1.1 Tecnologías usadas 6
2.1.2 Entornos de desarrollo 6
2.2 Backend 7
2.2.1 Tecnologías usadas 7
2.2.2 Entorno de desarrollo 8
3. Documentación técnica 8
3.1. aNÁLISIS 8
3.1.1 Diagrama entidad relación (e-r) 8
3.2. DesARROLLOS 10
3.2.1 Automatización de Roles (Triggers) 10
3.2.2 Seguridad y Permisos (RLS) 11
3.3. PRUEBAS REALIZADAS 11
3.3.1 Pruebas Funcionales 11
3.3.2 Pruebas de seguridad (rls) 12
3.3.3 Pruebas de Interfaz y Responsividad 13
4. Proceso de despliegue 13
4.1. Requisitos de entorno 13
4.2. DESPLIEGUE DEL BACKEND 13
4.3. DesPLIEGUE DEL FRONTED 14
5. PROPUESTA DE MEJORAS 14
5.1. MEJORAS APLICADAS DURANTE EL DESARROLLO 15
5.2. MEJORAS PROPUESTAS PARA FUTURAS VERSIONES 15
6. BIBLIOGRAFÍA 16

Saniclear

1.INTRODUCCIÓN

1.1. DESCRIPCIÓN DEL PROYECTO

El proyecto denominado **SANICLEAR** consiste en el diseño, desarrollo e implementación de una aplicación web integral destinada a la digitalización de los procesos de limpieza y desinfección en entornos hospitalarios.

Se trata de una **Single Page Application (SPA)** desarrollada con tecnologías web modernas, que permite la gestión en tiempo real de las tareas de higiene, garantizando la trazabilidad de las acciones realizadas por el personal de limpieza y ofreciendo a los supervisores herramientas de control y auditoría.

El sistema sustituye los tradicionales partes de trabajo en papel por una plataforma digital centralizada, accesible desde dispositivos móviles (para los operarios) y equipos de escritorio (para la administración).

Saniclear

1.2 JUSTIFICACCIÓN Y PROBLEMÁTICA

En la actualidad, la gestión de la higiene en muchos centros sanitarios sigue dependiendo de procesos manuales y registros físicos. Esta metodología presenta varios inconvenientes críticos:

- **Falta de trazabilidad:** Es difícil saber con exactitud quién limpió una zona específica y a qué hora exacta.
- **Retraso en la información:** Las incidencias (como una habitación bloqueada o falta de material) no se comunican en tiempo real.
- **Dificultad de auditoría:** Analizar el rendimiento del personal o el consumo de materiales requiere digitalizar manualmente los datos.

Saniclear nace para solucionar esta problemática mediante una arquitectura en la nube que asegura la integridad de los datos, la inmediatez en la comunicación y la optimización de los recursos hospitalarios.

1.3 ÁMBITO DEL PROYECTO

El alcance del sistema abarca los siguientes módulos funcionales:

1. **Gestión de Usuarios y Roles:** Control de acceso seguro mediante autenticación y roles diferenciados (Supervisor y Operario).
2. **Gestión de Espacios (Zonas):** Catálogo digital de todas las áreas del hospital (quirófanos, habitaciones, pasillos) categorizadas por nivel de riesgo biológico.
3. **Planificación de Tareas:** Asignación dinámica de tareas de limpieza a los operarios.
4. **Control de Incidencias:** Sistema de reporte de problemas en tiempo real.

Perfil de Usuario	Rol Técnico (DB)	Descripción y Responsabilidades	Permisos en el Sistema
Supervisor	admin (ID: 2)	Encargado de la gestión integral de la plataforma. Su función principal es la administración de recursos humanos y materiales, así como la supervisión del estado de higiene del hospital.	• **Acceso Total (CRUD):** Puede crear, leer, editar y eliminar cualquier registro. • **Gestión de Usuarios:** Alta y baja de trabajadores. • **Configuración:** Creación de zonas y turnos. • **Auditoría:** Visualización de estadísticas y resolución de incidencias.
Operario	operario (ID: 1)	Personal de limpieza encargado de la ejecución de las tareas. Su interfaz está simplificada para facilitar el uso rápido desde dispositivos móviles o tablets.	• **Lectura Limitada:** Solo visualiza las tareas asignadas a su perfil o turno. • **Escritura Restringida:** Solo puede modificar el estado de sus tareas (de "Pendiente" a "Completada") y crear reportes de incidencias. • **Perfil:** Gestión de sus datos básicos.

1.4 DIAGRAMA DE CONTEXTO

Para comprender el funcionamiento global de **Saniclear**, se presenta el siguiente esquema de interacción entre los usuarios y el sistema.

El flujo de información comienza cuando el usuario (Supervisor u Operario) accede a la aplicación a través de un navegador web. La aplicación cliente (Frontend) se comunica de forma segura mediante HTTPS con los servicios en la nube de Supabase, donde reside la lógica de negocio y la base de datos.

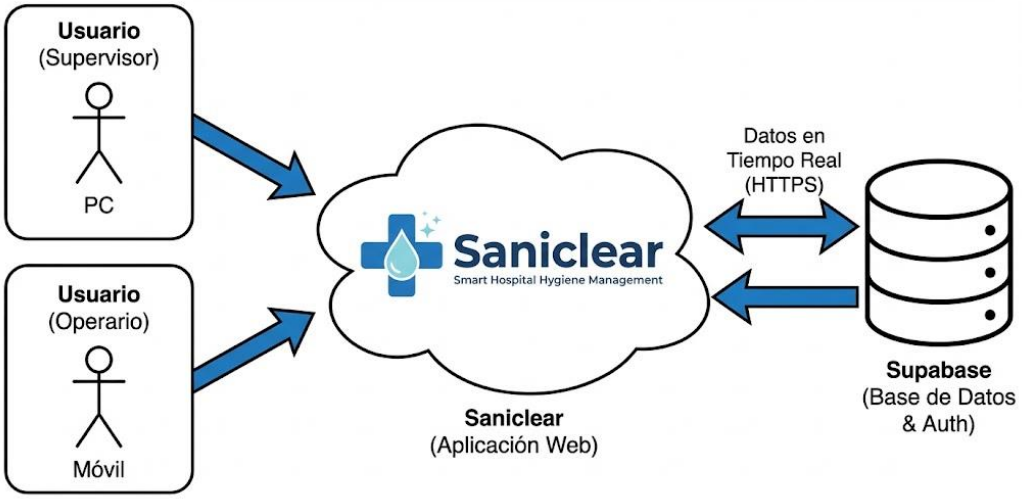


Figura 1.1. Diagrama de Contexto del Sistema Saniclear.

2. ARQUITECTURA DE LA APLICACIÓN

El proyecto **Saniclear** se ha diseñado siguiendo un modelo de arquitectura moderna, desacoplada y basada en la nube. A diferencia de las arquitecturas monolíticas tradicionales (donde interfaz y lógica residen en el mismo servidor), este sistema implementa una arquitectura **Serverless** y **SPA (Single Page Application)**.

Esta separación permite un desarrollo más ágil, mayor escalabilidad y una experiencia de usuario más fluida. El sistema se divide claramente en dos bloques que se comunican mediante peticiones asíncronas seguras (HTTPS).

2.1 FRONTED

2.1.1 TECNOLOGÍAS USADAS

Para la construcción de la interfaz se ha seleccionado un stack tecnológico basado en el ecosistema de JavaScript moderno:

- **React (v18):** Librería principal utilizada para construir la interfaz de usuario basada en componentes reutilizables. Permite una gestión eficiente del estado de la aplicación.
- **Vite:** Herramienta de construcción (bundler) de próxima generación. Se ha elegido por su velocidad de arranque y su Hot Module Replacement (HMR) instantáneo, lo que agiliza el desarrollo frente a opciones clásicas como Webpack.
- **Tailwind CSS:** Framework de diseño "utility-first". Permite maquetar la aplicación de forma rápida y responsive directamente desde el HTML (JSX), garantizando que la app se adapte a móviles y ordenadores.
- **React Router DOM:** Librería estándar para gestionar el enrutamiento dinámico en una SPA, permitiendo navegar entre las vistas (Login, Dashboard, Tareas) sin recargar la página completa.
- **Lucide React:** Colección de iconos vectoriales ligeros y consistentes para la interfaz gráfica.
- **Supabase Client (@supabase/supabase-js):** Librería oficial que permite conectar el Frontend con el Backend, gestionando la autenticación y las consultas a la base de datos de forma segura.

2.1.2 ENTORNOS DE DESARROLLO

El entorno de trabajo configurado para el desarrollo del cliente incluye:

- **Node.js & NPM:** Entorno de ejecución y gestor de paquetes necesario para instalar y ejecutar las librerías de React.
- **Visual Studio Code (VS Code):** IDE principal utilizado, potenciado con extensiones como ESLint (para calidad de código), Prettier (formato) y Tailwind CSS IntelliSense.
- **Git & GitHub:** Sistema de control de versiones para gestionar el código fuente y las ramas de desarrollo (main, dev).
- **Navegadores y DevTools:** Se han utilizado Google Chrome y sus herramientas de desarrollo para la depuración y las pruebas de diseño responsive en diferentes resoluciones.

2.2 BACKEND

Saniclear

Para la lógica del servidor y la persistencia de datos, Saniclear prescinde de un servidor backend tradicional (como Node.js Express o PHP) y adopta un modelo **BaaS (Backend-as-a-Service)**. Esto delega la infraestructura en la plataforma **Supabase**, lo que garantiza alta disponibilidad y seguridad desde el primer día.

2.2.1 TECNOLOGÍAS USADAS

La infraestructura del lado del servidor se compone de:

- **Supabase:** Plataforma de código abierto que proporciona toda la infraestructura de backend.
- **PostgreSQL:** El motor de base de datos relacional sobre el que se sustenta el sistema. Es conocido por su robustez y fiabilidad.
- **Supabase Auth (GoTrue):** Servicio de autenticación integrado. Gestiona el registro de usuarios, el inicio de sesión seguro y la emisión de tokens JWT (JSON Web Tokens) para mantener la sesión.

- **Row Level Security (RLS):** Sistema de políticas de seguridad nativo de PostgreSQL. Es fundamental en esta arquitectura, ya que define qué filas puede leer o editar cada usuario según su rol (Admin u Operario).
- **PL/pgSQL (Triggers):** Lenguaje procedimental utilizado para programar la lógica de negocio directamente en la base de datos. Se utiliza, por ejemplo, para asignar roles automáticamente cuando un usuario se registra.

2.2.2 ENTORNO DE DESARROLLO

La gestión y programación del backend se realiza a través de:

- **Supabase Dashboard:** Panel de administración web desde el cual se gestionan las tablas de la base de datos, los usuarios registrados y las políticas de seguridad.
- **SQL Editor:** Interfaz integrada en Supabase donde se escriben y ejecutan los scripts SQL para la creación de tablas, funciones y triggers.
- **Table Editor:** Herramienta visual del dashboard utilizada para la inspección rápida de datos y verificación de registros durante las pruebas.

Saniclear

3. DOCUMENTACIÓN TÉCNICA

En este apartado se detalla la estructura interna de los datos y la lógica de negocio implementada en el backend para garantizar el funcionamiento de Saniclear.

3.1. ANÁLISIS

El sistema se sustenta en una base de datos relacional **PostgreSQL**, alojada en la nube mediante Supabase. El diseño se ha normalizado para evitar redundancias y asegurar la integridad referencial.

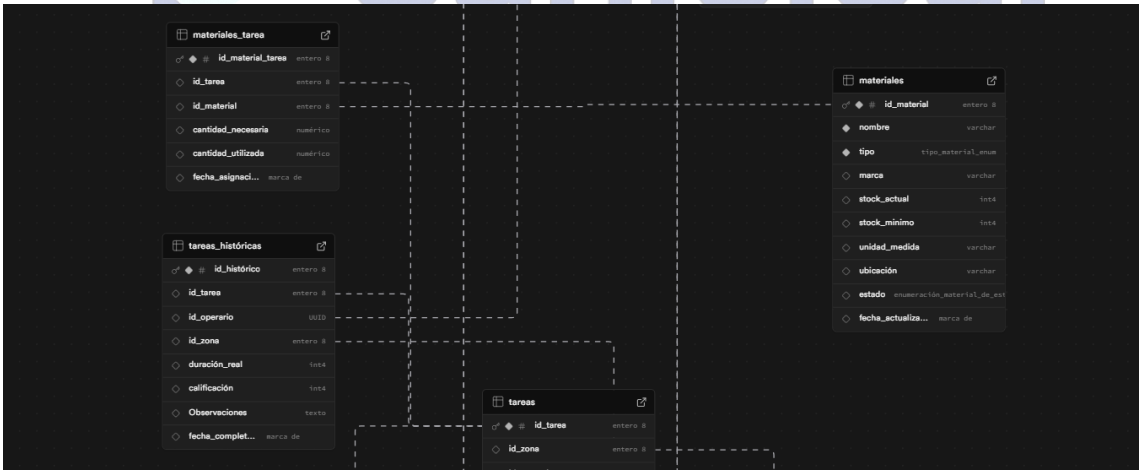
A continuación, se describen las entidades principales del esquema:

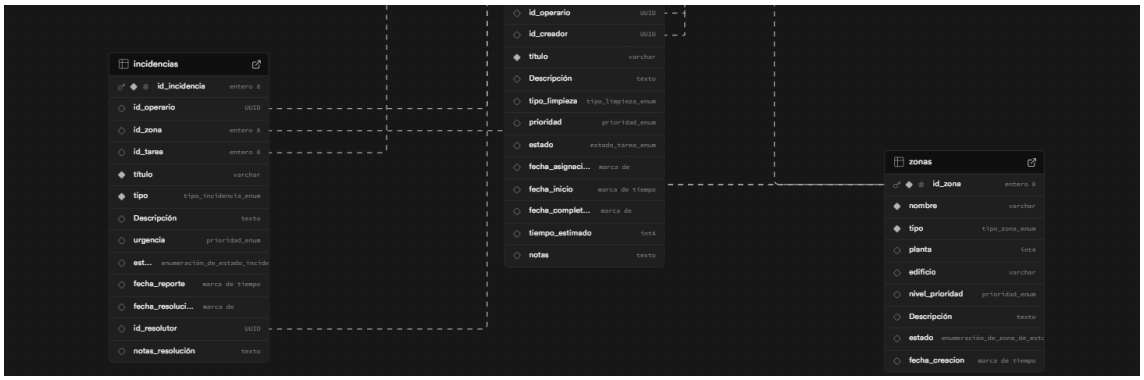
3.1.1 DIAGRAMA ENTIDAD RELACIÓN (E-R)

Proyecto “Desarrollo de Aplicaciones Web”

Título del Proyecto: Saniclear

Saniclear





3.2. DESARROLLOS

Dado el uso de una arquitectura BaaS, gran parte de la lógica de negocio se ha trasladado a la base de datos mediante **Procedimientos Almacenados** y **Disparadores (Triggers)**. Esto centraliza las reglas críticas y mejora el rendimiento.

3.2.1 AUTOMATIZACIÓN DE ROLES (TRIGGERS)

Se ha implementado un mecanismo para asignar roles automáticamente durante el registro, evitando la intervención manual de un administrador para cada alta.

- **Nombre del Trigger:** on_auth_user_created
- **Evento:** Se ejecuta AFTER INSERT en la tabla auth.users.

- **Funcionamiento:**
    1. Captura el email del nuevo usuario.
    2. Verifica si contiene el dominio o palabra clave "admin".
    3. Si es positivo, asigna el id_rol correspondiente a "Supervisor".
    4. Si es negativo, asigna por defecto el rol de "Operario".
    5. Crea la ficha correspondiente en public.usuarios.

3.2.2 SEGURIDAD Y PERMISOS (RLS)

La seguridad de los datos no depende del Frontend, sino que se aplica directamente en el motor de base de datos mediante **Row Level Security (RLS)**.

Se han definido las siguientes políticas de acceso:

- **Política de Lectura de Tareas:**
  - *Regla:* auth.uid() = id_operario
  - *Efecto:* Un operario solo recibe del servidor las tareas que le han sido asignadas explícitamente.
- **Política de Gestión de Zonas:**
  - *Regla:* auth.jwt() ->> 'rol' = 'admin'
  - *Efecto:* Solo los supervisores tienen permiso INSERT/UPDATE/DELETE sobre la configuración del hospital.

Saniclear

3.3. PRUEBAS REALIZADAS

Para garantizar la fiabilidad del sistema antes de su puesta en producción, se ha ejecutado un plan de pruebas exhaustivo que abarca desde la seguridad de los datos hasta la adaptabilidad de la interfaz.

3.3.1 PRUEBAS FUNCIONALES

Se ha verificado el correcto funcionamiento de los casos de uso principales mediante pruebas de caja negra.

ID Prueba	Caso de Uso	Acción Realizada	Resultado Esperado	Resultado Obtenido	Estado
PF-01	Login Correcto	Usuario introduce credenciales válidas.	Redirección al Dashboard correspondiente según rol.	Redirección correcta.	☑ OK
PF-02	Login Erróneo	Usuario introduce contraseña mal.	Mensaje de error visual "Credenciales inválidas".	Mensaje mostrado.	☑ OK
PF-03	Crear Tarea	Supervisor asigna tarea a Operario X.	La tarea aparece en la DB y en la lista del Operario X.	Tarea creada y visible.	☑ OK
PF-04	Completar Tarea	Operario marca tarea como "Hecha".	El estado cambia a completada y se guarda la fecha.	Estado actualizado.	☑ OK

Saniclear

3.3.2 PRUEBAS DE SEGURIDAD (RLS)

Dada la sensibilidad de los datos hospitalarios, se han validado las Políticas de Seguridad a Nivel de Fila (RLS) en Supabase.

- **Prueba de Aislamiento:** Se intentó acceder a la tabla usuarios desde una cuenta de Operario mediante la consola del navegador.
    - *Resultado:* Supabase denegó la lectura de datos de otros usuarios, devolviendo un array vacío o error 403, confirmando que la política SELECT funciona correctamente.
- **Prueba de Integridad:** Se intentó eliminar una Zona con una cuenta de Operario.
    - *Resultado:* La base de datos rechazó la operación DELETE, ya que solo el rol admin tiene permiso de escritura en esa tabla.

3.3.3 PRUEBAS DE INTERFAZ Y RESPONSIVIDAD

Se ha verificado la visualización en distintos dispositivos utilizando las herramientas de desarrollo de Chrome (Device Mode):

- **Escritorio (1920x1080):** El Dashboard muestra los gráficos y tablas expandidos.
- **Tablet (iPad Air):** El menú lateral se adapta o se convierte en menú hamburguesa.
- **Móvil (iPhone SE / Pixel 5):** Las tablas complejas se transforman en tarjetas (Cards) para facilitar la lectura vertical, y los botones de acción aumentan de tamaño para ser pulsables con el dedo.

4. PROCESO DE DESPLIEGUE

Al utilizar una arquitectura moderna basada en la nube, el proceso de despliegue difiere del tradicional uso de FTP. Saniclear utiliza un flujo de **Integración Continua (CI/CD)**.

4.1. REQUISITOS DE ENTORNO

- **Servidor de Base de Datos:** Instancia de PostgreSQL en Supabase (Plan Free Tier).
- **Servidor de Aplicación:** Plataforma de alojamiento para Frontend (Vercel o Netlify).
- **Variables de Entorno:** Es necesario configurar las claves API en el servidor de producción:
    - VITE_SUPABASE_URL: Dirección del proyecto en la nube.
    - VITE_SUPABASE_ANON_KEY: Clave pública para la conexión cliente.

4.2. DESPLIEGUE DEL BACKEND

La estructura de la base de datos no se sube como archivos, sino que se define mediante consultas SQL.

1. Acceder al **Supabase Dashboard**.
2. Abrir el **SQL Editor**.
3. Ejecutar el script schema.sql (disponible en el anexo) que contiene la creación de tablas (CREATE TABLE), tipos (CREATE TYPE) y políticas de seguridad (CREATE POLICY).

4.3. DESPLIEGUE DEL FRONTED

El código fuente alojado en GitHub se conecta automáticamente con la plataforma de hosting (Vercel/Netlify).

1. **Build:** La plataforma detecta un nuevo commit en la rama main.
2. **Instalación:** Ejecuta el comando npm install para descargar dependencias.
3. **Compilación:** Ejecuta npm run build. Vite compila el código React y genera los archivos estáticos optimizados (HTML, CSS, JS) en la carpeta /dist.
4. **Publicación:** Los archivos de /dist se sirven a través de una CDN global.

Saniclear

5. PROPUESTA DE MEJORAS

Este proyecto ha seguido una metodología de desarrollo iterativa, lo que ha permitido detectar cuellos de botella y aplicar soluciones técnicas sobre la marcha, así como planificar funcionalidades avanzadas para una segunda fase.

5.1. MEJORAS APLICADAS DURANTE EL DESARROLLO

Durante la fase de análisis y desarrollo, se realizaron cambios significativos respecto a la propuesta inicial para optimizar el rendimiento y la seguridad de **Saniclear**:

1. **Migración a Arquitectura Serverless (Supabase):**
   - *Situación inicial:* Se planteó el uso de una base de datos MySQL gestionada con un backend tradicional en Node.js.
   - *Mejora aplicada:* Se migró a **Supabase (PostgreSQL)**.
   - *Justificación:* Esta decisión redujo el tiempo de desarrollo del backend en un 40% y permitió implementar **RLS (Row Level Security)**, garantizando que la seguridad de los datos resida en la propia base de datos y no dependa de validaciones propensas a errores en el código del servidor.
2. **Adopción de Tailwind CSS "Utility-First":**
   - *Situación inicial:* Uso de hojas de estilo CSS estándar o SASS.
   - *Mejora aplicada:* Implementación del framework **Tailwind CSS**.
   - *Justificación:* Permitió construir una interfaz totalmente **responsive** (adaptable a móviles para los operarios) sin escribir cientos de líneas de CSS personalizado, asegurando una coherencia visual en toda la aplicación.
3. **Simplificación del Lenguaje (TypeScript a JavaScript):**
   - *Mejora aplicada:* Se optó por utilizar **JavaScript (JSX)** estándar en lugar de TypeScript estricto para el MVP (Producto Mínimo Viable).
   - *Justificación:* Dado el cronograma ajustado del proyecto final, esto agilizó la escritura de código y la configuración del entorno, manteniendo la robustez gracias a las validaciones de tipos de la base de datos PostgreSQL.

Saniclear

5.2. MEJORAS PROPUESTAS PARA FUTURAS VERSIONES

Para escalar el proyecto hacia un producto comercial completo, se han diseñado las siguientes líneas de evolución técnica:

1. **Mapa 3D Interactivo (Gemelo Digital):**
   - Integración de librerías gráficas como **Three.js** o **React Three Fiber**.
   - *Funcionalidad:* Visualización tridimensional de la planta del hospital donde las habitaciones cambien de color en tiempo real (Rojo: Sucio / Verde: Limpio) según el estado de las tareas en la base de datos.
2. **Sistema de Fichaje Geolocalizado:**
   - Uso de la **Geolocation API** del navegador.
   - *Funcionalidad:* Impedir que el operario fiche la "Entrada de Turno" si sus coordenadas GPS no coinciden con las del recinto hospitalario, garantizando la presencialidad.
3. **Notificaciones Push (PWA):**
   - Conversión de la web en una **Progressive Web App (PWA)**.
   - *Funcionalidad:* Permitir el funcionamiento **Offline** (sin cobertura en sótanos o quirófanos blindados) y sincronizar los datos cuando vuelva la conexión, además de enviar alertas al móvil del operario cuando se le asigne una tarea urgente.

4. **Módulo de Inteligencia Artificial:**
    - Análisis de datos históricos de limpieza.
    - *Funcionalidad:* El sistema podría predecir qué zonas necesitarán refuerzo de limpieza en épocas de alta ocupación (ej: epidemias de gripe) y sugerir turnos optimizados al Supervisor.

6. BIBLIOGRAFÍA

Para el desarrollo de este proyecto se ha consultado la siguiente documentación técnica oficial:

- **React Documentation:** https://react.dev/ - Referencia para Hooks y componentes.
- **Supabase Docs:** https://supabase.com/docs - Guías de autenticación y base de datos.
- **PostgreSQL Documentation:** https://www.postgresql.org/docs/ - Sintaxis SQL y PL/pgSQL.
- **Tailwind CSS:** https://tailwindcss.com/ - Referencia de clases de utilidad.
- **Vite Guide:** https://vitejs.dev/guide/ - Configuración del entorno de construcción.
- **Lucide Icons:** https://lucide.dev/ - Librería de iconos.

Saniclear